Microsoft Technical Interview Questions & Answers

Computer Networks | Operating Systems | Object-Oriented Programming

---

📡 Computer Networks

Q1. What is the OSI Model? Explain its layers.

Answer: OSI (Open Systems Interconnection) Model ek reference model hai जो network communication को 7 layers में divide करता है:

1. **Physical Layer**: Actual hardware और electrical signals को handle करता है (cables, switches)
2. **Data Link Layer**: Node-to-node data transfer और error detection (MAC addresses, Ethernet)
3. **Network Layer**: Routing और logical addressing (IP addresses, routers)
4. **Transport Layer**: End-to-end communication और reliability (TCP/UDP, port numbers)
5. **Session Layer**: Sessions को establish और manage करता है
6. **Presentation Layer**: Data formatting, encryption/decryption
7. **Application Layer**: User applications और services (HTTP, FTP, SMTP)

याद रखने का तरीका: "Please Do Not Throw Sausage Pizza Away"

Q2. Difference between TCP and UDP?

Answer:

TCP (Transmission Control Protocol):

- Connection-oriented protocol hai (3-way handshake)
- Reliable delivery guarantee करता है
- Data packets को order में deliver करता है
- Slower but more reliable
- Use cases: Web browsing, email, file transfer

UDP (User Datagram Protocol):

- Connectionless protocol hai
- No guarantee of delivery
- Faster क्योंकि कोई overhead नहीं
- Packets order में नहीं आते necessarily
- Use cases: Video streaming, gaming, DNS queries

Q3. What is a subnet mask and why is it used?

Answer: Subnet mask ek 32-bit number है जो IP address को दो parts में divide करता है:

- **Network portion** (कौन सा network है)

- **Host portion** (network में कौन सा device है)

Example: IP = 192.168.1.10, Subnet Mask = 255.255.255.0

- पहले 24 bits network को identify करते हैं
- Last 8 bits host को identify करते हैं

Benefits:

- Network को smaller sub-networks में divide करने में मदद करता है
- Security और performance improve होती है
- IP addresses की efficient utilization होती है

Q4. Explain the TCP 3-way handshake.

Answer: यह TCP connection establish करने की process है:

1. **SYN**: Client server को SYN (synchronize) packet भेजता है
2. **SYN-ACK**: Server SYN-ACK (synchronize-acknowledgment) packet भेजता है
3. **ACK**: Client ACK (acknowledgment) packet भेजकर connection confirm करता है

इसके बाद data transfer शुरू होता है। यह ensure करता है कि दोनों sides ready हैं communication के लिए।

Q5. What is DNS and how does it work?

Answer: DNS (Domain Name System) internet की "phone book" है जो human-readable domain names को IP addresses में convert करती है।

Process:

1. आप browser में www.google.com type करते हैं
2. DNS resolver आपके ISP से query करता है
3. अगर cached नहीं है, तो root DNS server से पूछता है
4. फिर TLD (Top Level Domain) server (.com) से पूछता है
5. Finally authoritative DNS server actual IP address return करता है
6. Browser उस IP address पर connect करता है

Q6. Difference between HTTP and HTTPS?

Answer:

HTTP (HyperText Transfer Protocol):

- Port 80 use करता है
- Data plain text में transfer होता है
- Secure नहीं है
- Fast है क्योंकि कोई encryption नहीं

HTTPS (HTTP Secure):

- Port 443 use करता है
- SSL/TLS encryption use करता है
- Secure communication provide करता है
- थोड़ा slower है encryption की वजह से
- Authentication और data integrity ensure करता है

Q7. What is ARP (Address Resolution Protocol)?

Answer: ARP एक protocol है जो IP address को MAC (physical) address में convert करता है।

Example: जब आपका computer किसी दूसरे device से communicate करना चाहता है same network में:

- आपके पास destination का IP address है
- लेकिन actual data transfer के लिए MAC address चाहिए
- ARP broadcast message भेजता है: "Who has IP 192.168.1.5?"
- वो device reply करता है अपना MAC address के साथ
- यह mapping ARP cache में store हो जाती है

Q8. What are the different types of networks?

Answer:

1. **LAN (Local Area Network)**: छोटा area जैसे office या building (high speed, low latency)
2. **WAN (Wide Area Network)**: बड़ा geographical area जैसे cities या countries (Internet एक WAN है)
3. **MAN (Metropolitan Area Network)**: एक city या large campus को cover करता है
4. **PAN (Personal Area Network)**: बहुत छोटा area, personal devices (Bluetooth, USB)
5. **WLAN (Wireless LAN)**: Wireless technology use करके LAN (WiFi)

Q9. What is IP addressing? Explain IPv4 vs IPv6.

Answer: IP address एक unique identifier है जो network पर devices को identify करता है।

IPv4:

- 32-bit address (4 octets)
- Format: 192.168.1.1
- लगभग 4.3 billion addresses
- Dotted decimal notation
- Address space खत्म हो रहा है

IPv6:

- 128-bit address
- Format: 2001:0db8:85a3:0000:0000:8a2e:0370:7334
- 340 undecillion addresses (virtually unlimited)
- Hexadecimal notation
- Better security और routing

Q10. What is a Gateway and Router? Difference?

Answer:

Gateway:

- दो different networks को connect करता है जो different protocols use करते हैं
- Protocol conversion करता है
- OSI model की सभी layers पर काम करता है
- Example: Email gateway, VoIP gateway

Router:

- Same protocol वाले networks को connect करता है
- Packets को best path से forward करता है
- Network layer (Layer 3) पर काम करता है
- IP addresses use करके routing करता है

Simple difference: Gateway = protocol translator, Router = packet forwarder

---

💻 Operating Systems

Q11. What is an Operating System? What are its functions?

Answer: Operating System एक software है जो computer hardware और user के बीच interface provide करता है।

Main Functions:

1. **Process Management**: Processes को create, schedule, और terminate करना
2. **Memory Management**: RAM को allocate और deallocate करना
3. **File Management**: Files को organize, store, और retrieve करना
4. **Device Management**: Input/output devices को control करना
5. **Security**: Access control और protection provide करना
6. **User Interface**: Command line या GUI provide करना

Q12. Difference between Process and Thread?

Answer:

Process:

- Independent program in execution
- अपना separate memory space होता है
- Creating process expensive होता है (ज्यादा resources)
- Inter-process communication complex है
- एक process crash होने से दूसरे processes पर असर नहीं

Thread:

- Lightweight subprocess, process के अंदर चलता है

- Same memory space share करते हैं same process के threads
- Creating thread faster और cheaper है
- Threads आपस में easily communicate कर सकते हैं
- एक thread crash हो तो पूरा process crash हो सकता है

Q13. What is Deadlock? What are its conditions?

Answer: Deadlock एक situation है जहां दो या ज्यादा processes एक दूसरे के resources के लिए indefinitely wait कर रहे हों।

4 Necessary Conditions (सभी simultaneously होने चाहिए):

1. **Mutual Exclusion**: Resources को simultaneously share नहीं किया जा सकता
2. **Hold and Wait**: Process कुछ resources hold किए हुए है और और resources के लिए wait कर रहा है
3. **No Preemption**: Resources को forcefully नहीं छीना जा सकता
4. **Circular Wait**: Processes का circular chain है जहां हर process अगले की resource चाहता है

Example: Process P1 ने Resource R1 hold किया और R2 चाहिए, Process P2 ने R2 hold किया और R1 चाहिए - Deadlock!

Deadlock Prevention/Avoidance:

- एक या ज्यादा conditions को violate करो
- Banker's Algorithm use करो
- Resource allocation graph maintain करो

Q14. Explain different CPU Scheduling Algorithms.

Answer:

1. **FCFS (First Come First Serve)**:
    - जो पहले आया उसे पहले execute करो
    - Simple लेकिन convoy effect हो सकता है
    - Non-preemptive
    - Average waiting time ज्यादा हो सकता है
2. **SJF (Shortest Job First)**:
    - सबसे छोटा burst time वाला process पहले
    - Minimum average waiting time
    - Starvation हो सकता है long processes का
    - Non-preemptive version
3. **SRTF (Shortest Remaining Time First)**:
    - SJF का preemptive version
    - अगर नया process आए छोटे burst time के साथ तो current को preempt करो
4. **Round Robin**:

    - हर process को fixed time quantum मिलता है
    - Time quantum खत्म होने पर अगला process
    - Fair scheduling, preemptive
    - Context switching overhead होता है

5. **Priority Scheduling**:
    - Highest priority वाला process पहले execute होता है
    - Starvation problem (aging से solve करते हैं)
    - Preemptive या non-preemptive हो सकता है

Q15. What is Virtual Memory?

Answer: Virtual Memory एक technique है जो secondary storage (hard disk) को RAM की तरह use करती है, जिससे programs को लगता है कि unlimited memory available है।

Benefits:

- बड़े programs run कर सकते हैं limited RAM में
- Multiple programs simultaneously run कर सकते हैं
- Memory को efficiently utilize करते हैं
- Degree of multiprogramming बढ़ती है

Working:

- Program को pages में divide करते हैं
- जरूरत के pages RAM में load होते हैं
- बाकी pages disk पर रहते हैं
- Page fault होने पर disk से RAM में transfer होता है
- Page table maintain करते हैं mapping के लिए

Q16. Difference between Paging and Segmentation?

Answer:

Paging:

- Memory को fixed-size blocks (pages) में divide करते हैं
- Internal fragmentation हो सकता है
- Hardware support चाहिए (page table)
- User को invisible है
- Faster memory access
- Simple implementation

Segmentation:

- Program को logical units (segments) में divide करते हैं जैसे functions, arrays
- External fragmentation हो सकता है
- Segment table use करते हैं

- User-visible है (programmer segments define कर सकता है)
- More flexible लेकिन complex
- Better से logical organization reflect करता है

Q17. What is Thrashing?

Answer: Thrashing एक condition है जब system ज्यादा time paging (swapping pages) में spend करता है actual execution की बजाय।

Causes:

- बहुत सारे processes simultaneously run हो रहे हैं
- Insufficient RAM available है
- High degree of multiprogramming
- Poor page replacement algorithm

Effects:

- CPU utilization drastically कम हो जाता है
- System performance बहुत slow हो जाती है
- Processes के execution time बढ़ जाता है
- Disk I/O बहुत बढ़ जाती है

Solution:

- Reduce degree of multiprogramming
- Increase RAM
- Use better page replacement algorithms
- Working set model implement करो
- Local page replacement use करो

Q18. Explain different Page Replacement Algorithms.

Answer:

1. **FIFO (First In First Out)**:
    - जो page सबसे पहले आया उसे पहले replace करो
    - Simple लेकिन Belady's Anomaly हो सकता है
    - Queue data structure use करते हैं
2. **LRU (Least Recently Used)**:
    - जो page सबसे लंबे time से use नहीं हुआ उसे replace करो
    - Better performance लेकिन implementation complex
    - Counter या stack maintain करना पड़ता है
    - Most practical algorithm
3. **Optimal Page Replacement**:

- जो page सबसे दूर future में use होगा उसे replace करो
- Best algorithm (lowest page fault rate)
- Practical नहीं क्योंकि future predict नहीं कर सकते
- Benchmark के लिए use होता है

4. **LFU (Least Frequently Used)**:
   - जो page सबसे कम बार use हुआ उसे replace करो
   - Counter maintain करना पड़ता है

Q19. What is a Semaphore? Binary vs Counting Semaphore.

Answer: Semaphore एक synchronization tool है जो multiple processes/threads के बीच shared resources की access को control करता है।

Types:

1. **Binary Semaphore (Mutex)**:
   - Values: 0 या 1
   - Mutual exclusion के लिए use होता है
   - एक time पर सिर्फ एक process critical section में

2. **Counting Semaphore**:
   - Values: 0 से N तक
   - Multiple instances of resources को manage करने के लिए
   - Example: 5 printers available हैं तो semaphore = 5

Operations:

- **wait() / P() / down()**: Semaphore value decrease करता है, 0 हो तो wait करता है
- **signal() / V() / up()**: Semaphore value increase करता है, waiting processes को wake up करता है

Example:

```
semaphore mutex = 1;

wait(mutex);
// Critical Section
signal(mutex);
```

Q20. Difference between Kernel Mode and User Mode?

Answer:

User Mode:

- Normal applications run होती हैं

- Limited access to system resources
- Hardware directly access नहीं कर सकते
- Safe execution environment
- Crash होने पर सिर्फ application crash होती है

Kernel Mode:

- OS kernel run होता है
- Complete access to all hardware और memory
- Privileged instructions execute कर सकते हैं
- Device drivers run होते हैं
- Crash होने पर पूरा system crash हो सकता है

Mode Switching: जब user program को hardware access चाहिए, तो system call करके kernel mode में switch होता है।

Q21. What is Context Switching?

Answer: Context Switching वो process है जब CPU एक process से दूसरे process पर switch करता है।

Steps:

1. Current process की state save करो (PCB में)
2. Registers, program counter, stack pointer save करो
3. Next process की state load करो
4. Execution resume करो

Context में क्या save होता है:

- Program Counter (PC)
- CPU registers
- Memory management information
- Process state
- I/O status

Overhead: Context switching expensive operation है क्योंकि:

- Time लगता है state save/load करने में
- Cache flush हो जाती है
- TLB (Translation Lookaside Buffer) flush होती है

Q22. What is Inter-Process Communication (IPC)?

Answer: IPC mechanisms हैं जिनसे processes आपस में communicate और data share करते हैं।

Methods:

1. **Shared Memory**:
    - Processes एक common memory area share करते हैं

- Fast communication
- Synchronization की जरूरत होती है

2. **Message Passing**:
   - Processes messages send/receive करते हैं
   - send() और receive() operations
   - No synchronization issues
   - Slower than shared memory
3. **Pipes**:
   - Unidirectional communication channel
   - Parent-child processes के बीच
   - Named pipes (FIFO) bidirectional हो सकते हैं
4. **Sockets**:
   - Network-based communication
   - Different machines पर processes communicate कर सकते हैं
5. **Signals**:
   - Asynchronous notifications
   - Limited data transfer

🎯 Object-Oriented Programming (OOPS)

Q23. What are the four pillars of OOP?

Answer:

1. **Encapsulation (Data Hiding)**:
   - Data और methods को एक unit (class) में bind करना
   - Data को direct access से protect करना (private variables)
   - Public methods के through ही access होता है
   - Data integrity maintain होती है

```
class BankAccount {
    private double balance;  // hidden

    public void deposit(double amount) {
        if(amount > 0)
            balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

2. **Inheritance (विरासत)**:
   - Existing class की properties को new class में reuse करना
   - Code reusability बढ़ता है
   - IS-A relationship establish होता है

```
class Animal {
    void eat() { }
}
class Dog extends Animal {
    void bark() { }
}  // Dog IS-A Animal
```

3. **Polymorphism (बहुरूपता)**:
   - Same method different forms में काम करे
   - "Many forms" का मतलब
   - **Compile-time (Static)**: Method Overloading
   - **Runtime (Dynamic)**: Method Overriding

```
// Compile-time
void print(int a) { }
void print(String a) { }

// Runtime
Animal a = new Dog();
a.sound();  // Dog's sound() will be called
```

4. **Abstraction**:
   - Complex implementation को hide करके sirf essential features दिखाना
   - "What" दिखाना, "How" छुपाना
   - Abstract classes और interfaces use करते हैं

```
abstract class Vehicle {
    abstract void start();  // how? नहीं बताया
}
```

Q24. Difference between Abstract Class and Interface?

Answer:

Feature	Abstract Class	Interface

Feature	Abstract Class	Interface
Methods	Abstract + Concrete दोनों	सभी abstract (Java 8+ में default भी)
Variables	Any type हो सकते हैं	सिर्फ public static final constants
Constructor	हो सकते हैं	नहीं हो सकते
Inheritance	Single (एक ही extend)	Multiple (कई implement)
Keyword	`extends`	`implements`
Access Modifiers	Any modifier	सभी methods public by default
Speed	Fast	Slow (extra indirection)

जब Use करें:

- **Abstract Class**: जब common behavior है और state maintain करना है, partial implementation provide करना है
- **Interface**: जब sirf contract define करना है बिना implementation के, multiple inheritance चाहिए

Example:

```
// Abstract Class
abstract class Animal {
    String name;  // state
    abstract void sound();
    void sleep() { }  // concrete method
}

// Interface
interface Flyable {
    void fly();  // contract
}

class Bird extends Animal implements Flyable {
    void sound() { }
    public void fly() { }
}
```

Q25. Explain Method Overloading vs Method Overriding.

Answer:

Method Overloading (Compile-time Polymorphism):

- Same class में same name के multiple methods
- Parameters different होने चाहिए (number, type, या order)
- Return type different हो सकता है लेकिन काफी नहीं
- Static binding होती है (compile time पर decide)

- Same class या subclass में हो सकता है

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method Overriding (Runtime Polymorphism):

- Parent class के method को child class में redefine करना
- Method signature same होना चाहिए (name, parameters, return type)
- Dynamic binding होती है (runtime पर decide)
- Inheritance चाहिए
- `@Override` annotation use करते हैं (optional but recommended)

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

Animal a = new Dog();
a.sound();  // Output: Bark (runtime पर decide होता है)
```

Key Differences:

- Overloading = Same method, different parameters
- Overriding = Same method signature, different implementation

---

Q26. What is Constructor? Types of Constructors?

Answer: Constructor एक special method है जो object create होने पर automatically call होता है।

Characteristics:

- Class के नाम जैसा ही नाम होता है
- कोई return type नहीं होता (void भी नहीं)
- Object initialization के लिए use होता है
- Overloaded हो सकता है

Types:

1. **Default Constructor**:
    - कोई parameters नहीं
    - Compiler automatically provide करता है अगर आप नहीं बनाते
    - सभी variables को default values से initialize करता है

```
class Student {
    String name;
    int age;

    Student() {  // default
        name = "Unknown";
        age = 0;
    }
}
```

2. **Parameterized Constructor**:
    - Parameters accept करता है
    - Different values से object initialize कर सकते हैं
    - Multiple constructors हो सकते हैं (overloading)

```
class Student {
    String name;
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }
}

Student s = new Student("Raj", 20);
```

3. **Copy Constructor**:
    - Existing object से new object create करता है
    - Java में explicitly define करना पड़ता है

```
class Student {
    String name;
    int age;

    // Copy constructor
    Student(Student s) {
        this.name = s.name;
        this.age = s.age;
    }
}

Student s1 = new Student("Raj", 20);
Student s2 = new Student(s1);  // copy
```

Q27. What is `this` keyword in Java/C++?

Answer: `this` एक reference variable है जो current object को point करता है।

Uses:

1. **Instance variables को refer करने के लिए** जब parameter का नाम same हो:

```
class Student {
    String name;

    Student(String name) {
        this.name = name;  // this.name = instance variable
        // name = parameter
    }
}
```

2. **Current class के method को call करने के लिए**:

```
class Student {
    void display() { }

    void show() {
        this.display();  // optional
    }
}
```

3. **Constructor chaining के लिए** (एक constructor से दूसरा call):

```
class Student {
    String name;
```

```
    int age;

    Student() {
        this("Default", 0);  // दूसरा constructor call
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

4. **Current object को return करने के लिए**:

```
class Student {
    Student getStudent() {
        return this;
    }
}
```

Note: Static methods में `this` use नहीं कर सकते क्योंकि static context में कोई current object नहीं होता।

Q28. Difference between `static` and `final` keywords?

Answer:

Static:

- Class level पर belong करता है, not to object
- सभी objects के लिए shared होता है
- Object बनाए बिना भी access कर सकते हैं
- Memory में एक ही बार allocate होता है (class loading के समय)

```
class Counter {
    static int count = 0;  // shared by all objects

    Counter() {
        count++;
    }

    static void display() {
        System.out.println("Count: " + count);
    }
}

Counter.display();  // no object needed
Counter c1 = new Counter();
```

```
Counter c2 = new Counter();
// count = 2 (shared)
```

Final:

- Constants define करने के लिए
- एक बार assign होने के बाद change नहीं हो सकता
- **Final variable**: constant value, re-assignment नहीं
- **Final method**: override नहीं हो सकता child class में
- **Final class**: inherit नहीं हो सकता

```
final double PI = 3.14159;  // constant
// PI = 3.14;  // Error!

final class Math {
    // कोई class इसे extend नहीं कर सकता
}

class Parent {
    final void display() {
        // override नहीं हो सकता
    }
}
```

Static Final (Together):

```
class Constants {
    static final double PI = 3.14159;
    // class-level constant
}
```

Q29. What is Exception Handling? Explain try-catch-finally.

Answer: Exception handling एक mechanism है runtime errors को gracefully handle करने के लिए, ताकि program crash न हो।

Structure:

```
try {
    // risky code जो exception throw कर सकता है
    int result = 10 / 0;  // ArithmeticException

} catch (ArithmeticException e) {
    // specific exception handle करो
    System.out.println("Cannot divide by zero: " + e.getMessage());
```

```
} catch (Exception e) {
    // general exception (सबसे last में)
    System.out.println("Error: " + e);

} finally {
    // हमेशा execute होगा (optional)
    // cleanup code - file close, connection close
    System.out.println("Finally block executed");
}
```

Types of Exceptions:

1. **Checked Exceptions**:
    - Compile-time पर check होते हैं
    - Handle करना compulsory है
    - Examples: IOException, SQLException, ClassNotFoundException
2. **Unchecked Exceptions**:
    - Runtime पर occur होते हैं
    - Handle करना optional है
    - Examples: NullPointerException, ArrayIndexOutOfBounds, ArithmeticException

Keywords:

- `try`: risky code
- `catch`: exception handle करने के लिए
- `finally`: cleanup code (हमेशा execute होगा)
- `throw`: manually exception throw करने के लिए
- `throws`: method signature में declare करने के लिए

Example with throw/throws:

```
void checkAge(int age) throws Exception {
    if(age < 18) {
        throw new Exception("Age must be 18+");
    }
}
```

Q30. What is the difference between `==` and `.equals()`?

Answer:

`==` Operator:

- Reference comparison करता है (memory address compare करता है)
- Primitive types के लिए values compare करता है
- दोनों objects same memory location पर हैं क्या?

- Faster operation

```
int a = 5, b = 5;
a == b;  // true (value comparison)

String s1 = new String("Hello");
String s2 = new String("Hello");
s1 == s2;  // false (different objects in heap)

String s3 = "Hello";
String s4 = "Hello";
s3 == s4;  // true (same object in string pool)
```

`.equals()` Method:

- Content comparison करता है (actual values compare करता है)
- Object class में defined है
- Override किया जा सकता है custom comparison के लिए
- String class में यह characters compare करता है

```
String s1 = new String("Hello");
String s2 = new String("Hello");
s1.equals(s2);  // true (same content)

// Custom class में override
class Student {
    String name;
    int age;

    @Override
    public boolean equals(Object obj) {
        if(this == obj) return true;
        if(obj == null || getClass() != obj.getClass())
            return false;

        Student s = (Student) obj;
        return age == s.age && name.equals(s.name);
    }
}
```

Important:

- हमेशा Strings के लिए `.equals()` use करें, न कि `==`
- Objects compare करने के लिए `.equals()` override करें
- `.equals()` override करते समय `hashCode()` भी override करें

Q31. What is Inheritance? Types of Inheritance.

Answer: Inheritance वो mechanism है जिससे एक class (child) दूसरी class (parent) की properties और methods को acquire कर सकती है।

Benefits:

- Code reusability
- Method overriding possible
- Polymorphism achieve कर सकते हैं
- Hierarchical classification

Types:

1. **Single Inheritance**: एक parent, एक child

```
class Animal { }
class Dog extends Animal { }
```

2. **Multilevel Inheritance**: Chain of inheritance

```
class Animal { }
class Dog extends Animal { }
class Puppy extends Dog { }
```

3. **Hierarchical Inheritance**: एक parent, multiple children

```
class Animal { }
class Dog extends Animal { }
class Cat extends Animal { }
```

4. **Multiple Inheritance**: एक child, multiple parents
    - Java classes में support नहीं (Diamond Problem)
    - Interfaces के through possible है

```
interface A { }
interface B { }
class C implements A, B { }
```

5. **Hybrid Inheritance**: Multiple types का combination
    - Java में partially supported (through interfaces)

Q32. What is Multiple Inheritance and why doesn't Java support it?

Answer: Multiple Inheritance का मतलब है एक class multiple parent classes से inherit करे।

Java में क्यों नहीं है: **Diamond Problem** की वजह से:

```
      Animal
     /      \
   Dog      Cat
     \      /
     PetDog
```

अगर Dog और Cat दोनों में `sound()` method है, तो PetDog को कौन सा use करना चाहिए? Ambiguity आ जाती है।

```
class Animal {
    void sound() { }
}

class Dog extends Animal {
    void sound() { System.out.println("Bark"); }
}

class Cat extends Animal {
    void sound() { System.out.println("Meow"); }
}

// अगर यह possible होता:
class PetDog extends Dog, Cat {
    // sound() कौन सा call होगा? Ambiguity!
}
```

Java का Solution:

- Classes के साथ multiple inheritance नहीं
- लेकिन **multiple interfaces implement** कर सकते हैं क्योंकि:
  - Interfaces में सिर्फ method declarations होते थे (Java 8 से पहले)
  - Implementation child class में होता है
  - Java 8+ में default methods के साथ भी explicitly resolve करना पड़ता है

```
interface A {
    void method();
}

interface B {
    void method();
}

class C implements A, B {
    public void method() {
        // एक ही implementation दोनों के लिए
```

```
        // No ambiguity
    }
}
```

Q33. What is Polymorphism? Explain with examples.

Answer: Polymorphism का मतलब है "many forms" - एक entity के multiple forms हो सकते हैं।

Types:

1. Compile-time Polymorphism (Static/Early Binding):

Method Overloading:

```
class Calculator {
    // Same method name, different parameters
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator c = new Calculator();
c.add(5, 10);        // calls first method
c.add(5.5, 10.5);    // calls second method
c.add(5, 10, 15);    // calls third method
```

Operator Overloading (C++ में, Java में नहीं):

```
// + operator for both addition and concatenation
int a = 5 + 10;           // addition
string s = "Hello" + " World";  // concatenation
```

2. Runtime Polymorphism (Dynamic/Late Binding):

Method Overriding:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
```

```
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

// Runtime polymorphism
Animal a1 = new Dog();
Animal a2 = new Cat();

a1.sound();  // Dog barks (runtime पर decide)
a2.sound();  // Cat meows
```

Real-world Example:

```
class Shape {
    void draw() { }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

// Polymorphic behavior
Shape s1 = new Circle();
Shape s2 = new Rectangle();

s1.draw();  // Drawing Circle
s2.draw();  // Drawing Rectangle
```

Q34. What is Access Modifier? Explain all types.

Answer: Access Modifiers वो keywords हैं जो class, method, या variable की accessibility को control करते हैं।

Types (Java में):

1. **Public**:
   - कहीं से भी access हो सकता है
   - कोई restriction नहीं
   - Any package से access possible

```
public class Student {
    public String name;
    public void display() { }
}
```

2. **Private**:
   - सिर्फ same class के अंदर access हो सकता है
   - Strongest restriction
   - Encapsulation के लिए best

```
class Student {
    private int age;

    private void calculate() { }

    // Getter/Setter for access
    public int getAge() { return age; }
}
```

3. **Protected**:
   - Same package में access possible
   - Different package में सिर्फ child classes access कर सकती हैं (inheritance के through)

```
class Parent {
    protected void method() { }
}

class Child extends Parent {
    void test() {
        method();  // accessible
    }
}
```

4. **Default (Package-Private)**:
    - कोई modifier नहीं लिखते तो default
    - सिर्फ same package में access possible
    - Different package से access नहीं

```
class Student {
    String name;  // default
    void display() { }  // default
}
```

Accessibility Table:

Modifier	Same Class	Same Package	Subclass	Everywhere
Public	✓	✓	✓	✓
Protected	✓	✓	✓	×
Default	✓	✓	×	×
Private	✓	×	×	×

Q35. What is the difference between Composition and Aggregation?

Answer: दोनों association के types हैं जो HAS-A relationship represent करते हैं।

Composition (Strong relationship):

- Strong ownership
- Parts container के बिना exist नहीं कर सकते
- Container destroy हो जाए तो parts भी destroy हो जाते हैं
- "Part-of" relationship

```
class Engine {
    // Engine details
}

class Car {
    private Engine engine;  // Car HAS-A Engine

    Car() {
        engine = new Engine();  // Car creates Engine
    }
    // अगर Car destroy हो तो Engine भी destroy
}
```

Example: House और Rooms - अगर House destroy हो जाए तो Rooms का कोई मतलब नहीं।

Aggregation (Weak relationship):

- Weak ownership
- Parts independently exist कर सकते हैं
- Container destroy हो जाए तो parts exist रहते हैं
- "Has-a" relationship

```
class Department {
    private List<Teacher> teachers;

    void addTeacher(Teacher t) {
        teachers.add(t);  // Teacher पहले से exist करता है
    }
    // Department destroy हो जाए तो Teachers exist रहते हैं
}

class Teacher {
    // Teacher independently exists
}
```

Example: Department और Teachers - अगर Department बंद हो जाए तो Teachers दूसरी departments में जा सकते हैं।

Key Difference:

- Composition = Cannot exist without container (Lifetime dependency)
- Aggregation = Can exist independently (No lifetime dependency)

🎓 Bonus Important Questions

Q36. What is the difference between Stack and Heap memory?

Answer:

Stack Memory:

- Method calls और local variables store होते हैं
- LIFO (Last In First Out) structure
- Automatically managed है
- Fast access
- Size limited है
- Thread-specific है (हर thread का अपना stack)
- Scope खत्म होने पर automatically deallocate

```
void method() {
    int x = 10;  // Stack में
    String s = "Hello";  // reference stack में
}
```

Heap Memory:

- Objects और instance variables store होते हैं
- Dynamically allocated
- Slow access (compared to stack)
- Size बड़ी है
- सभी threads share करते हैं
- Garbage collector manage करता है

```
Student s = new Student();  // Object heap में, reference stack में
```

Memory Allocation:

```
void createStudent() {
    Student s = new Student();
    // s (reference) → Stack
    // new Student() (object) → Heap
}
```

Q37. What is Garbage Collection in Java?

Answer: Garbage Collection automatic memory management है जो unused objects को heap memory से remove करता है।

Working:

- JVM automatically unused objects को identify करता है
- जो objects कोई reference नहीं रखते उन्हें garbage मानता है
- Periodically garbage collector run होता है और memory free करता है

Example:

```
Student s1 = new Student();
s1 = null;  // अब पहला object unreachable है
// Garbage collector इसे remove कर देगा

Student s2 = new Student();
Student s3 = new Student();
s2 = s3;  // अब s2 का पुराना object unreachable
```

Benefits:

- Manual memory management की जरूरत नहीं
- Memory leaks कम होते हैं

- Programmer को memory management की चिंता नहीं

Methods:

- `System.gc()` - garbage collector को request करना (guarantee नहीं)
- `finalize()` - object destroy होने से पहले call होता है

Q38. Explain Object-Oriented Design Principles (SOLID).

Answer:

S - Single Responsibility Principle: एक class की एक ही responsibility होनी चाहिए।

```
// Bad
class Student {
    void calculateGrade() { }
    void saveToDatabase() { }  // different responsibility
}

// Good
class Student {
    void calculateGrade() { }
}
class StudentRepository {
    void saveToDatabase() { }
}
```

O - Open/Closed Principle: Class extension के लिए open, modification के लिए closed।

```
// Use inheritance/interfaces instead of modifying existing code
```

L - Liskov Substitution Principle: Subclass parent class की जगह use हो सके बिना program break किए।

```
Animal a = new Dog();  // Dog can replace Animal
```

I - Interface Segregation Principle: बड़ी interfaces को छोटी specific interfaces में break करो।

```
// बेहतर है multiple छोटी interfaces
interface Flyable { }
interface Swimmable { }
```

D - Dependency Inversion Principle: High-level modules को low-level modules पर depend नहीं करना चाहिए।

```
// Depend on abstractions, not concrete classes
```

📚 Tips for Interview Success

Preparation Strategy:

1. **हर concept को examples के साथ समझें**
2. **Code लिखकर practice करें** - theory पढ़ना काफी नहीं
3. **Real-world applications सोचें** - जैसे banking system में encapsulation
4. **Differences clearly समझें** - TCP vs UDP, Process vs Thread
5. **Diagrams बना सकें** - OSI model, Deadlock, Memory layout
6. **Time complexity discuss करें** - scheduling algorithms के लिए

During Interview:

1. **Question को clearly समझें** पहले
2. **Loud thinking करें** - approach explain करें
3. **Examples दें** अपने answers में
4. **Edge cases mention करें**
5. **Follow-up questions के लिए ready रहें**
6. **अगर नहीं पता तो honestly बोलें**, guessing से बेहतर है

Common Follow-up Areas:

- "Can you write code for this?"
- "What will be the time complexity?"
- "Where is this used in real applications?"
- "What are the advantages and disadvantages?"

🎯 Final Notes

यह document covers:

- ✅ 38+ Most Important Questions
- ✅ Computer Networks (10 Questions)
- ✅ Operating Systems (12 Questions)
- ✅ OOP Concepts (16 Questions)
- ✅ Hindi + English Mixed Format
- ✅ Detailed Explanations with Examples
- ✅ Code Examples where relevant

Remember: Interview में confidence important है। अगर किसी question का answer नहीं पता तो honestly बोलें कि आप नहीं जानते, लेकिन सीखने को eager हैं।

All the Best for your Microsoft Interview! 🚀

Document created for Microsoft Technical Interview Preparation Topics: Computer Networks, Operating Systems, Object-Oriented Programming